

## **Disciplina: Sistemas Digitais**

**Professor:** Luciano Lores Caimi

# **RELATÓRIO 1: Somadores Rápidos**

**Rhuan Leandro Paiva**

|                                                     |          |
|-----------------------------------------------------|----------|
| <b>Disciplina: Sistemas Digitais</b>                | <b>1</b> |
| <b>Rhuan Leandro Paiva</b>                          | <b>1</b> |
| <b>Introdução</b>                                   | <b>3</b> |
| <b>2. Fundamentação Teórica</b>                     | <b>3</b> |
| 2.1 Somador Ripple Carry (RP)                       | 3        |
| 2.2 Somador Carry Select (CS)                       | 3        |
| 2.3 Subtração em Complemento de 2                   | 3        |
| 2.4 Overflow em Complemento de 2                    | 3        |
| <b>3. Implementação</b>                             | <b>4</b> |
| 3.1 Estrutura Modular                               | 4        |
| 3.2 Full Adder                                      | 4        |
| 3.3 Bloco RCA4                                      | 4        |
| 3.4 Bloco Carry Select                              | 4        |
| 3.5 Somador de 32 bits                              | 5        |
| <b>4. Análise do Caminho Crítico</b>                | <b>5</b> |
| 4.1 Atraso das portas                               | 5        |
| 4.2 Caminho Crítico do Ripple Carry (32 bits)       | 5        |
| 4.3 Caminho Crítico do Carry Select                 | 6        |
| 4.4 Comparação                                      | 6        |
| <b>5. Testes e Resultados</b>                       | <b>6</b> |
| 5.1 Propagação de Carry                             | 6        |
| a) $S = -1 + 1$                                     | 6        |
| b) $S = \text{AAAAAAA}_{16} + \text{5555555}_{16}$  | 6        |
| c) $S = \text{FF00FF0}_{16} - \text{00FF00FF}_{16}$ | 6        |
| 5.2 Overflow / Underflow                            | 7        |
| a)                                                  | 7        |
| b)                                                  | 7        |
| <b>6. Conclusão</b>                                 | <b>7</b> |

# RELATÓRIO – SOMADOR CARRY SELECT 32 BITS

## Introdução

Este trabalho tem como objetivo a implementação de um somador rápido de 32 bits do tipo Carry Select (CS) utilizando o Logisim Evolution. O circuito foi desenvolvido de forma modular, a partir de blocos de somadores de 4 bits, conforme solicitado no enunciado. Além da operação de soma, o circuito também realiza subtração utilizando complemento de dois e detecta a ocorrência de overflow/underflow.

## 2. Fundamentação Teórica

### 2.1 Somador Ripple Carry (RP)

O somador Ripple Carry realiza a soma propagando o carry sequencialmente de um bit para o próximo. Isso implica que o tempo de propagação cresce linearmente com o número de bits.

### 2.2 Somador Carry Select (CS)

O somador Carry Select melhora o desempenho calculando previamente dois resultados para cada bloco: um assumindo  $C_{in} = 0$  e outro assumindo  $C_{in} = 1$

Um multiplexador (MUX) seleciona o resultado correto quando o carry real é conhecido. Isso reduz o tempo de propagação, pois evita a espera completa do carry.

### 2.3 Subtração em Complemento de 2

A subtração é realizada por:  $Sub$  e  $C_{in} = \neg Sub$

$$A - B = A + (\neg B + 1)$$

No circuito:  $O$  é invertido usando XOR com  $Sub$  e  $C_{in} = \neg Sub$

### 2.4 Overflow em Complemento de 2

O overflow é detectado por:

$$Overflow = C_{31} \oplus C_{32}$$

Onde o  $C_{31}$ : carry antes do bit mais significativo e  $C_{32}$ : carry final

## 3. Implementação

### 3.1 Estrutura Modular

O circuito foi dividido em:

- Full Adder (1 bit)
- RCA4 (4 bits)
- Carry Select (4 bits)
- main Somador final de 32 bits

### 3.2 Full Adder

O Full Adder realiza:

$$S = A \oplus B \oplus Cin$$

$$Cout = AB + Cin(A \oplus B)$$

### 3.3 Bloco RCA4

O RCA4 é composto por 4 Full Adders em série. Além disso, foi implementado o cálculo de overflow interno:

$$Overflow = C3 \oplus C4$$

### 3.4 Bloco Carry Select

Cada bloco contém:

- dois RCA4:
  - um com  $Cin = 0$
  - outro com  $Cin = 1$
- multiplexadores para selecionar:
  - soma
  - carry
  - overflow

### 3.5 Somador de 32 bits

O circuito final é composto por 8 blocos Carry Select de 4 bits. O primeiro bloco utiliza o Cin real e os demais utilizam MUX para seleção

O overflow final é obtido a partir do último bloco.

## 4. Análise do Caminho Crítico

### 4.1 Atraso das portas

| Porta            | Atraso |
|------------------|--------|
| NOT              | 1 UT   |
| AND (2 entradas) | 2 UT   |
| OR (2 entradas)  | 2 UT   |
| XOR (2 entradas) | 3 UT   |
| MUX 2:1          | 3 UT   |

### 4.2 Caminho Crítico do Ripple Carry (32 bits)

TRP = Time Ripple Carry.

TFA = Time Full Adder.

TCS = Time Carry Select.

TRCA4 = Time Ripple Carry Adder 4 bits.

TMUX = Time Multiplexer.

UT = Unidade de Tempo.

Cada Full Adder depende do anterior:

$$\text{TRP} = 32 \times \text{TFA}$$

Considerando: XOR = 3 UT      AND/OR = 2 UT

A propagação do carry passa por aproximadamente:

$$\text{TFA} \approx 2 + 2 = 4 \text{ UT}$$

$$\text{TRP} \approx 32 \times 4 = 128 \text{ UT}$$

### 4.3 Caminho Crítico do Carry Select

Para o Carry Select o primeiro bloco: atraso de um RCA4 e os demais blocos: atraso dominado por MUX

$$TCS \approx TRCA4 + (nblocos - 1) \times TMUX$$

$$TRCA4 \approx 4 \times 4 = 16 \text{ UT}$$

$$TCS \approx 16 + 7 \times 3 = 37 \text{ UT}$$

### 4.4 Comparação

| Tipo         | Atraso  |
|--------------|---------|
| Ripple Carry | ~128 UT |
| Carry Select | ~37 UT  |

O Carry Select é significativamente mais rápido.

## 5. Testes e Resultados

### 5.1 Propagação de Carry

a)  $S = -1 + 1$

Resultado:

$$-1 + 1 = 0$$

Sem overflow.

b)  $S = \text{AAAAAAAA}_{16} + \text{55555555}_{16}$

Esses valores geram alternância de bits, provocando propagação intensa de carry.

c)  $S = \text{FF00FF00}_{16} - \text{00FF00FF}_{16}$

Testa propagação em blocos e comportamento na subtração.

## 5.2 Overflow / Underflow

a)

$80000000_{16} + FFFFFFFF_{16}$

Resultado fora do intervalo → overflow.

b)

$7FFFFFFF_{16} + 00000001_{16}$

Resultado fora do intervalo → overflow.

## 6. Conclusão

O somador Carry Select apresentou desempenho significativamente superior ao Ripple Carry, reduzindo o tempo de propagação do carry. A implementação modular facilitou o desenvolvimento e a validação do circuito. O uso do complemento de dois permitiu a realização de subtração sem circuitos adicionais. A detecção de overflow foi corretamente implementada, garantindo a validade dos resultados para operações com números com sinal.